

데이터 과학 및 분석

보강#2 - Numpy

호서대학교 조학수



01 Python

02 Numpy

03 Pandas

04 matplotlib

05 seaborn / plotly



02 Numpy



1 주요 파이썬 라이브러리

■ 주요 파이썬 라이브러리

패키지명	설명
NumPy	고성능의 과학계산 컴퓨팅과 데이터 분석에 필요한 기본 패키지
Pandas	고수준의 자료 구조와 파이썬을 통한 빠르고 쉬운 데이터 분석 도구 포함
Matplotlib	Numpy를 기반으로 도면을 그리는 라이브러리
Seaborn	Matplotlib을 기반으로한 통계 시각화 라이브러리
Plotly	독립적인 시각화 라이브러리로 대화형 그래프 및 3D, 지도 시각화 지원
Scikit-learn	머신러닝 학습/예측에 필요한 다양한 기능을 제공하는 패키지
TensorFlow	머신러닝(특히 딥러닝)에 특화된 프레임워크(by Google)
PyTorch / Keras	머신러닝(특히 딥러닝)에 특화된 프레임워크(by Facebook)



2 Numpy 소개

■ 넘파이의 개념

- 파이썬의 과학 계산을 위한 라이브러리
- 벡터나 행렬 같은 선형대수의 표현법을 코드로 처리
- 사실상의 표준 라이브러리
- 다차원 리스트나 크기가 큰 데이터 처리에 유리

■ 넘파이의 특징

- 속도가 빠르고 메모리 사용이 효율적
 - 데이터를 메모리에 할당하는 방식이 기존과 다름
- 반복문을 사용하지 않음
 - 연산할 때 병렬로 처리
 - 함수를 한 번에 많은 요소에 적용
- 다양한 선형대수 관련 함수 제공
- C, C++, 포트란 등 다른 언어와 통합 사용 가능

$$2x_1 + 2x_2 + x_3 = 9$$

$$2x_1 - x_2 + 2x_3 = 6$$

$$x_1 - x_2 + 2x_3 = 5$$

(a) 간단한 수식 문제

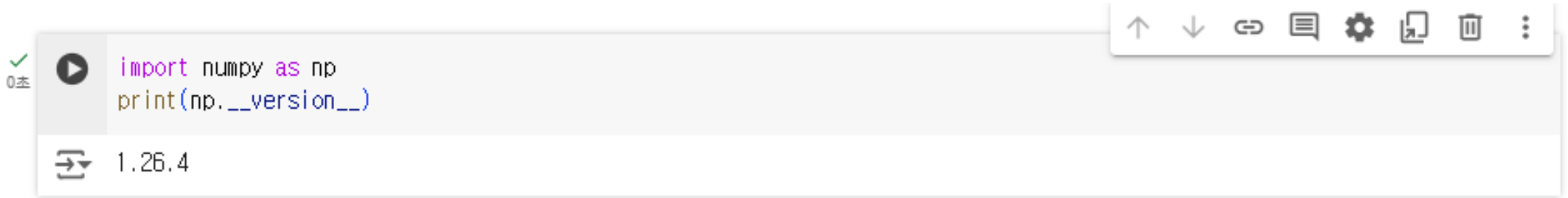
$$\begin{bmatrix} 2 & 2 & 1 & 9 \\ 2 & -1 & 2 & 6 \\ 1 & -1 & 2 & 5 \end{bmatrix}$$

(b) 행렬

그림 3-1 선형대수의 예시

2 Numpy 소개

■ NumPy 설치 확인



The screenshot shows a Jupyter Notebook interface. At the top right, there is a toolbar with icons for undo, redo, insert cell below, insert cell above, insert code cell, insert markdown cell, settings, copy, delete, and a menu. Below the toolbar, a code cell is displayed with a green checkmark and a play button icon on the left. The code inside the cell is:


```
import numpy as np
print(np.__version__)
```

 Below the code, the output is shown as '1.26.4' with a copy icon to its left.

- 위 코드로 numpy를 포함시키고 버전 확인
- 코드를 코랩에 복사 붙여넣기 할 땐, Ctrl + Shift + C,V로 가능

3 배열 - array

■ 배열 생성



#라이브러리로

```
import numpy as np
```

#다차원배열 생성

#벡터: 1차원배열, 행렬: 2차원배열, 그 이후는 다차원 배열

```
mat = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
```

mat



```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

3 배열 - array

- float32 데이터 타입인 넘파이 배열의 각 값을 저장하는 메모리 블록은 4바이트씩 차지

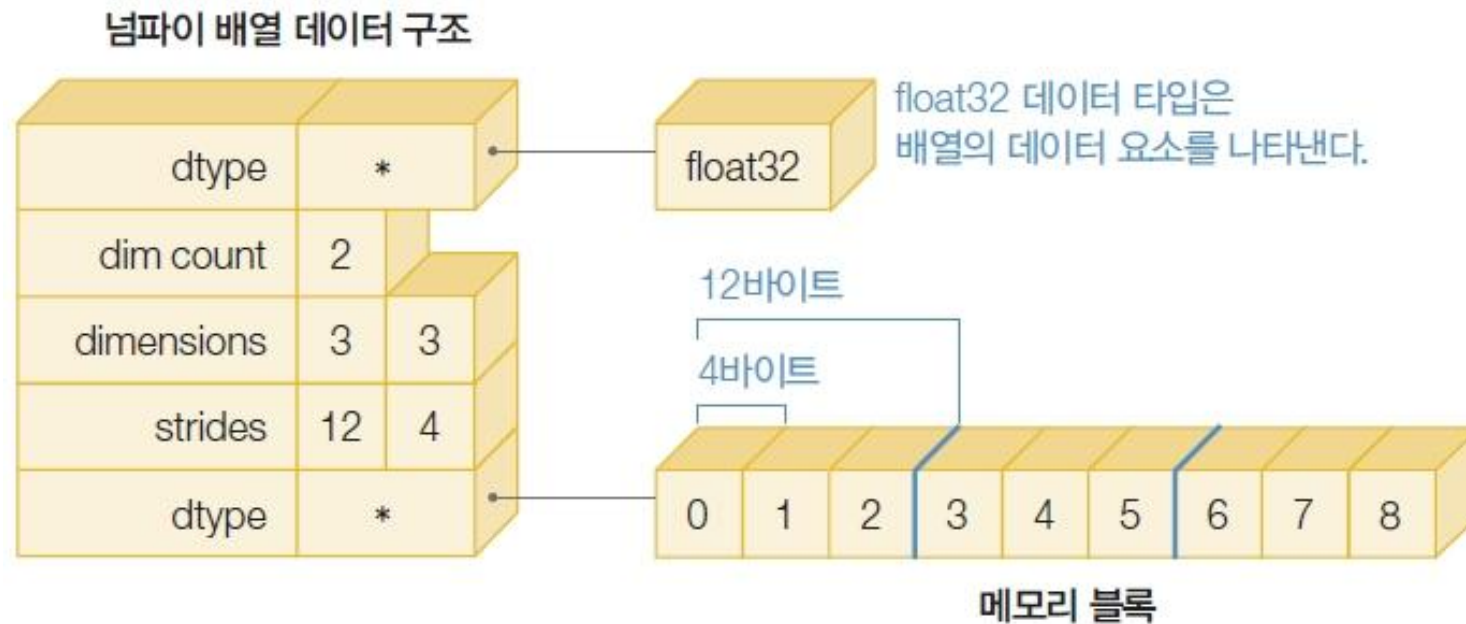


그림 3-3 넘파이 배열의 메모리 블록 배열

3 배열 - array

■ 배열 생성

```
▶ test_array = np.array([1, 4, 5, "8"], float)
   print(test_array)
```

```
↩ [1. 4. 5. 8.]
```

- 배열을 실수형으로 선언
- 배열을 출력해보면 값이 모두 실수형

```
▶ print(type(test_array[3]))
   # 실수형(floating Type)으로 자동 형 변환 실시
```

```
↩ <class 'numpy.float64'>
```

- 개별 값 데이터 타입도 <class 'numpy.float64'>로 출력
- float64는 64비트(bit), 즉 8바이트의 실수형 데이터



3 배열 - array

■ 배열 생성

▶ `print(test_array.dtype)` # 배열 전체의 데이터 타입 반환

↪ float64

▶ `print(test_array.shape)` # 배열의 구조(shape)를 반환함

↪ (4,)

- 데이터 특징을 출력하는 요소(property)는 dtype과 shape
- dtype은 넘파이 배열의 데이터 타입을 반환
- shape는 넘파이 배열에서 객체(object)의 차원(dimension)에 대한 구성 정보를 반환



3 배열 - array

■ 배열의 구조(shape)



그림 3-4 랭크가 1일 때 배열의 구조

```
▶ matrix = [[1, 2, 5, 8], [1, 2, 5, 8], [1, 2, 5, 8]]
np.array(matrix, int).shape
```

⇒ (3, 4)

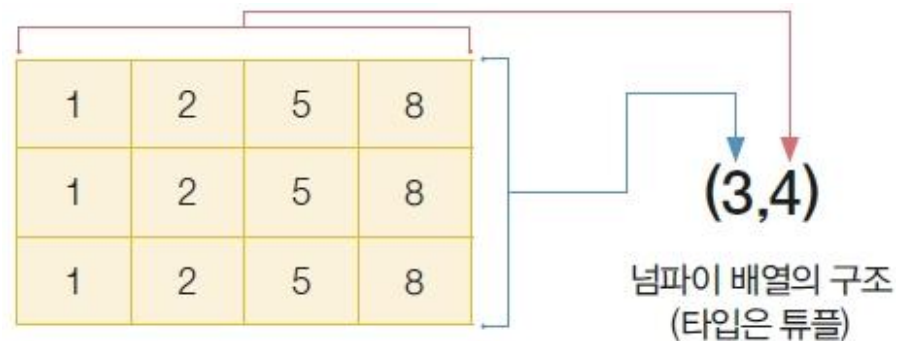


그림 3-5 랭크가 2일 때 배열의 구조



3 배열 - array

■ 배열의 구조(shape)

```
▶ tensor_rank3 = [
    [[1, 2, 5, 8], [1, 2, 5, 8], [1, 2, 5, 8]],
    [[1, 2, 5, 8], [1, 2, 5, 8], [1, 2, 5, 8]],
    [[1, 2, 5, 8], [1, 2, 5, 8], [1, 2, 5, 8]],
    [[1, 2, 5, 8], [1, 2, 5, 8], [1, 2, 5, 8]]
]
np.array(tensor_rank3, int).shape
```

↪ (3, 4)

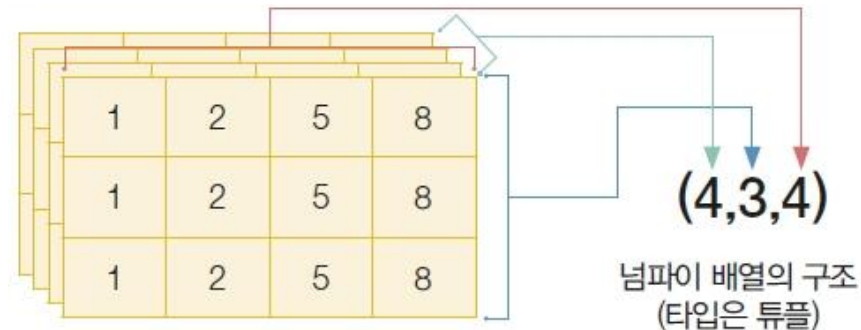


그림 3-6 랭크가 3일 때, 3차원 배열의 구조

3 배열 - array

■ dtype

- 매개변수 dtype으로 넘파이 배열의 데이터 타입 지정
- 변수가 사용하는 메모리 크기가 정해짐

```
▶ np.array( [ [1, 2, 3.5], [4, 5, 6.5]], dtype=int)
```

```
↪ array([[1, 2, 3],  
        [4, 5, 6]])
```

- dtype을 실수형인 float으로 지정한다면 모든 데이터가 실수형으로 저장되는 것을 확인 가능

```
▶ np.array( [ [1, 2, 3.5], [4, 5, 6.5]], dtype=float)
```

```
↪ array([[1. , 2. , 3.5],  
        [4. , 5. , 6.5]])
```

3 배열 - array

■ Dtype

```
▶ import sys
   np.array( [[1, 2, 3.5], [4, 5, 6.5]], dtype=np.float64).itemsize
```

↔ 8

```
▶ np.array( [[1, 2, 3.5], [4, 5, 6.5]], dtype=np.float32).itemsize
```

↔ 4

- itemsize 요소(property)로 넘파이 배열에서 각 요소가 차지하는 바이트(byte) 확인
- np.float64로 dtype을 선언하면 64비트, 즉 8바이트
- np.float32로 dtype을 선언하면 32비트, 즉 4바이트



3 배열 - array

■ 배열의 구조 다루기

- **reshape()** 함수로 배열의 구조를 변경하고 랭크를 조절

```
▶ x = np.array([[1, 2, 5, 8], [1, 2, 5, 8]])
x.shape
```

```
↔ (2, 4)
```

```
▶ x.reshape(-1,)
```

```
↔ array([1, 2, 5, 8, 1, 2, 5, 8])
```

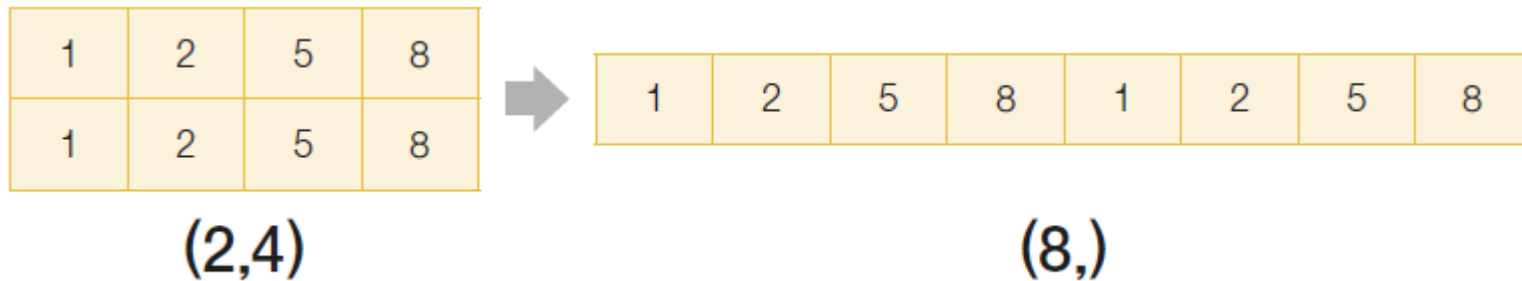


그림 3-7 reshape 함수로 변경된 배열의 구조

3 배열 - array

■ 배열의 구조 다루기

- -1을 사용하면 나머지 차원의 크기를 지정했을 때 전체 요소의 개수를 고려하여 마지막 차원이 **자동으로 지정**

```
▶ x = np.array(range(8)).reshape(4,2)
x
```

```
↔ array([[0, 1],
        [2, 3],
        [4, 5],
        [6, 7]])
```

```
▶ x.reshape(2,-1)
```

```
↔ array([[0, 1, 2, 3],
        [4, 5, 6, 7]])
```

```
▶ x.reshape(2,2,-1)
```

```
↔ array([[[0, 1],
         [2, 3]],
        [[4, 5],
         [6, 7]]])
```


3 배열 - array

- **flatten()** 함수는 데이터 그대로 1차원으로 변경
 - 데이터의 개수는 그대로 존재
 - 배열의 구조만 변한다

```
▶ x = np.array(range(8)).reshape(2,2,2)
x
```

```
↻ array([[[0, 1],
          [2, 3]],
        [[4, 5],
          [6, 7]]])
```

```
▶ x.flatten()
```

```
↻ array([0, 1, 2, 3, 4, 5, 6, 7])
```

0	1	4	5
2	3	6	7

(2,2,2)



0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

(8,)

그림 3-8 flatten 함수로 변경된 배열의 구조

3 배열 - array

■ 인덱싱(indexing) : 리스트에 있는 값의 상대적인 주소(offset)로 값에 접근

- 넘파이 배열의 인덱스 표현에는 ','을 지원
 - '[행][열]' 또는 '[행,열]' 형태
- 3차원 텐서 이상은 shape에서 출력되는 랭크 순서대로 인덱싱에 접근

```
▶ x = np.array([[1, 2, 3], [4, 5, 6]], int)
x
```

```
⇒ array([[1, 2, 3],
         [4, 5, 6]])
```

```
▶ x[0][0]
```

```
⇒ np.int64(1)
```

```
▶ x[0,2]
```

```
⇒ np.int64(3)
```

3 배열 - array

- 슬라이싱(slicing) : 인덱스를 사용하여 리스트 일부를 잘라내어 반환
 - 넘파이 배열은 행과 열을 나눠 슬라이싱할 수 있음

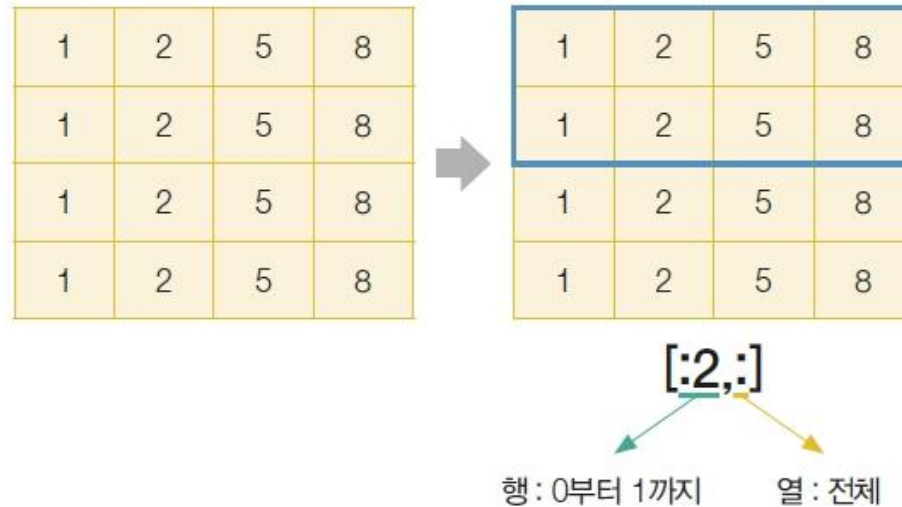


그림 3-9 슬라이싱

3 배열 - array

```
x = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]], int)
▶ x[:,2:] # 전체 행의 2열 이상
```

```
⇒ array([[ 3,  4,  5],
         [ 8,  9, 10]])
```

```
▶ x[1,1:3] # 1행의 1열 ~ 2열
```

```
⇒ array([7, 8])
```

```
▶ x[1:3] # 1행 ~ 2행의 전체
```

```
⇒ array([[ 6,  7,  8,  9, 10]])
```

3 배열 - array

■ 증가값(step) : 리스트에서 데이터의 요소를 호출할 때 데이터를 건너뛰면서 반환

- '[시작 인덱스:마지막 인덱스:증가값]' 형태
- 각 랭크에 있는 요소별로 모두 적용할 수 있음

```
x = np.array(range(15), int).reshape(3, -1)
x
```

```
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14]])
```

```
x[:, ::2]
```

```
array([[ 0,  2,  4],
       [ 5,  7,  9],
       [10, 12, 14]])
```

```
x[:, ::2, ::3]
```

```
array([[ 0,  3],
       [10, 13]])
```

4 배열 함수

■ 배열 생성 함수

■ arange

- range 함수와 같이 차례대로 값을 생성
- '(시작 인덱스, 마지막 인덱스, 증가값)'으로 구성
- range 함수와 달리 증가값에 실수형이 입력되어도 값을 생성할 수 있음
- 소수점 값을 주기적으로 생성할 때 유용

▶ np.arange(10)

⇒ array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

▶ np.arange(-5, 5)

⇒ array([-5, -4, -3, -2, -1, 0, 1, 2, 3, 4])

▶ np.arange(0, 5, 0.5)

⇒ array([0. , 0.5, 1. , 1.5, 2. , 2.5, 3. , 3.5, 4. , 4.5])

- 증가값에 실수형이 입력되어도 값을 생성
- 소수점 값을 주기적으로 생성할 때 유용

4 배열 함수

■ ones, zeros, empty

- ones 함수 : 1로만 구성된 넘파이 배열을 생성
 - 사전에 shape 값을 넣어서 원하는 크기의 넘파이 배열 생성
- zeros 함수 : 0으로만 구성된 넘파이 배열을 생성
- empty 함수 : 활용 가능한 메모리 공간 확보하여 반환
 - ones와 zeros는 먼저 shape의 크기만큼 메모리를 할당하고 그곳에 값을 채움
 - 해당 메모리 공간에 값이 남았을 경우 초기화 없이 그대로 반환
 - empty는 메모리 초기화 않아 생성될 때마다 다른 값 반환
- 생성 시점에서 dtype을 지정해주면 해당 데이터 타입으로 배열 생성

4 배열 함수

■ ones, zeros, empty

▶ `np.ones(shape=(5,2), dtype=np.int8)`

⇒ `array([[1, 1],
[1, 1],
[1, 1],
[1, 1],
[1, 1]], dtype=int8)`

▶ `np.zeros(shape=(2,2), dtype=np.float32)`

⇒ `array([[0., 0.],
[0., 0.]], dtype=float32)`

▶ `np.empty(shape=(2,4), dtype=np.float32)`

⇒ `array([[0. , 1.875 , 0. , 2.25],
[0. , 2.3125, 0. , 2.5]], dtype=float32)`



4 배열 함수

■ ones_like, zeros_like, empty_like

- ones_like 함수 : 기존 넘파이 배열과 같은 크기로 내용을 1로 채움
- zeros_like 함수 : 기존 넘파이 배열과 같은 크기로 내용을 0으로 채움
- empty_like 함수 : 기존 넘파이 배열과 같은 크기로 초기화 안된 상태로 만들

```
▶ x = np.arange(12).reshape(3,4)
x
```

```
⇒ array([[ 0,  1,  2,  3],
         [ 4,  5,  6,  7],
         [ 8,  9, 10, 11]])
```

```
▶ np.ones_like(x)
```

```
⇒ array([[1, 1, 1, 1],
         [1, 1, 1, 1],
         [1, 1, 1, 1]])
```

```
▶ np.zeros_like(x)
```

```
⇒ array([[0, 0, 0, 0],
         [0, 0, 0, 0],
         [0, 0, 0, 0]])
```

4 배열 함수

■ identity, eye, diag

- **identity** 함수 : 단위행렬(i행렬)을 생성
 - 매개변수 n 으로 $n \times n$ 단위행렬을 생성

▶ `np.identity(n=3, dtype=int)`

```
⇒ array([[1, 0, 0],
         [0, 1, 0],
         [0, 0, 1]])
```

▶ `np.identity(n=4, dtype=int)`

```
⇒ array([[1, 0, 0, 0],
         [0, 1, 0, 0],
         [0, 0, 1, 0],
         [0, 0, 0, 1]])
```

4 배열 함수

identity, eye, diag

- **eye** 함수 : 시작점과 행렬 크기를 지정, **단위행렬** 생성
 - N은 행의 개수, M은 열의 개수를 지정
 - k는 열의 값을 기준으로 시작 인덱스

▶ np.eye(3)

⇒ array([[1, 0, 0],
 [0, 1, 0],
 [0, 0, 1]])

▶ np.eye(N=3, M=5)

⇒ array([[1., 0., 0., 0., 0.],
 [0., 1., 0., 0., 0.],
 [0., 0., 1., 0., 0.]])

▶ np.eye(N=3, M=5, k=2)

⇒ array([[0., 0., 1., 0., 0.],
 [0., 0., 0., 1., 0.],
 [0., 0., 0., 0., 1.]])

4 배열 함수

identity, eye, diag

- diag** 함수 : 행렬의 대각성분만 array로 추출하거나 array로부터 대각 행렬 생성

`numpy.diag(v, k=0)`

$k > 0$ 이면 대각행렬 위의 성분, $k < 0$ 대각 행렬 아래의 성분

```
▶ arr = np.array([[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]])
diag_elements = np.diag(arr)
print(diag_elements)
```

⇒ [1 5 9]

```
▶ np.diag(arr, k=1)
```

⇒ array([2, 6])

```
▶ arr = np.array([1, 2, 3])
diag_matrix = np.diag(arr)
print(diag_matrix)
```

⇒ $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$

0	1	2
3	4	5
6	7	8

(a) `np.diag(matrix)`

0	1	2
3	4	5
6	7	8

(b) `np.diag(matrix, k=1)`

그림 3-11 diag 함수 사용

5 연산 함수

■ 통계 분석 함수

- **uniform** 함수 : **균등분포** 함수
 - 'np.random.uniform(시작값, 끝값, 데이터개수)'

▶ np.random.uniform(0, 5, 10)

```
↻ array([4.46911197, 0.76960499, 2.03737981, 1.04776323, 3.83270216,
        0.74273704, 4.53062972, 0.29589473, 3.13902949, 0.09892116])
```

- **normal** 함수 : **정규분포** 함수
 - 'np.random.normal(평균값, 분산, 데이터개수)'

▶ np.random.normal(0, 2, 10)

```
↻ array([-4.14472197, -1.61116466, -0.30634923, -2.48967782, -0.19921301,
        -0.98891176,  0.59887596,  2.2314008 , -1.61057297,  3.2466855 ])
```

5 연산 함수

■ 연산 함수

- 연산 함수(operation function) : 배열 내부 연산을 지원하는 함수
- 축(axis) : 배열의 랭크가 증가할 때마다 새로운 축이 추가되어 차원 증가
- **sum** 함수 : 각 요소의 합을 반환

```
▶ test_array = np.arange(1, 11)
   test_array.sum()
```

```
↔ np.int64(55)
```



5 연산 함수

■ 연산 함수

- sum 함수를 랭크가 2 이상인 배열에 적용할 때 축으로 연산의 방향을 설정

```
▶ test_array = np.arange(1,13).reshape(3,4)
test_array
```

```
↪ array([[ 1,  2,  3,  4],
        [ 5,  6,  7,  8],
        [ 9, 10, 11, 12]])
```

```
▶ test_array.sum(axis=0)
```

```
↪ array([15, 18, 21, 24])
```

```
▶ test_array.sum(axis=1)
```

```
↪ array([10, 26, 42])
```

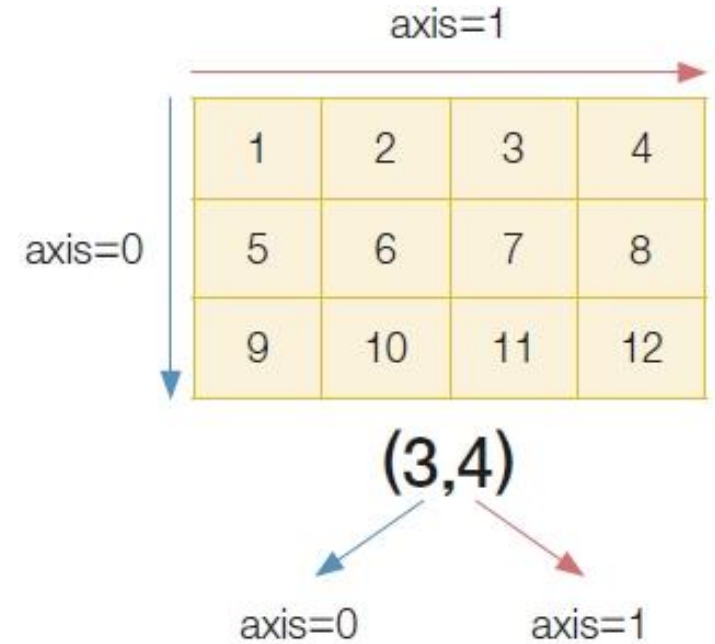


그림 3-12 축에 따른 연산

5 연산 함수

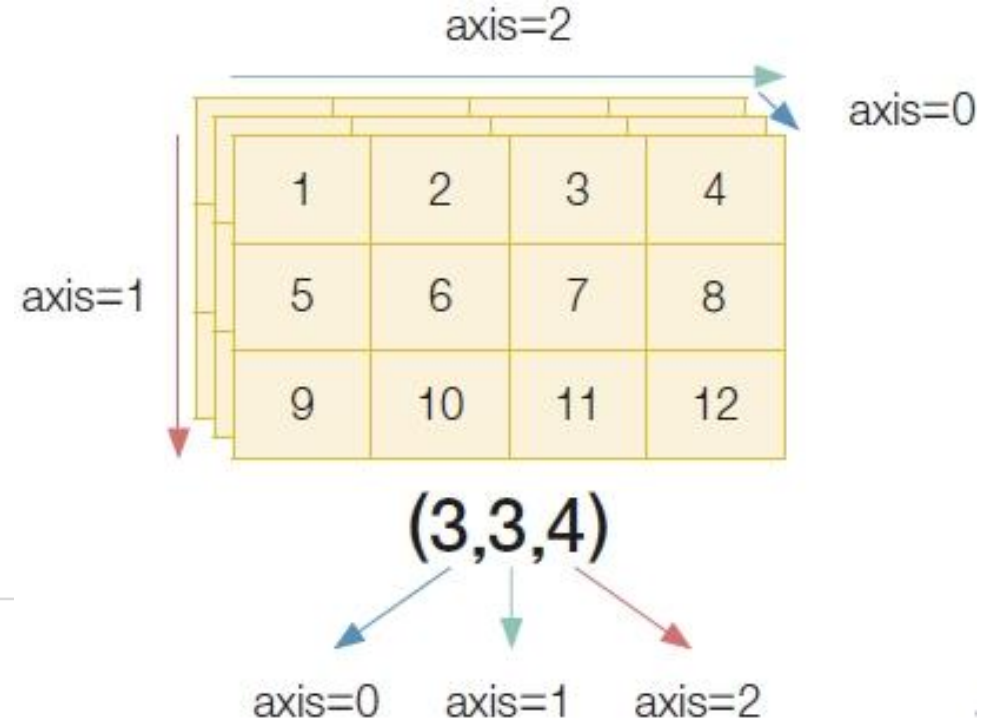
- 축별로 sum 함수 적용을 위한 (3,3,4) 행렬 생성

```
▶ test_array = np.arange(1, 13).reshape(3, 4)
  third_order_tensor = np.array([test_array, test_array, test_array])
  third_order_tensor
```

```
↻ array([[[ 1,  2,  3,  4],
          [ 5,  6,  7,  8],
          [ 9, 10, 11, 12]],

        [[ 1,  2,  3,  4],
          [ 5,  6,  7,  8],
          [ 9, 10, 11, 12]],

        [[ 1,  2,  3,  4],
          [ 5,  6,  7,  8],
          [ 9, 10, 11, 12]]])
```



5 연산 함수

- 축별로 sum 함수 적용 (3,3,4) 행렬 생성
 - axis 값 0, 1, 2 사례

▶ `third_order_tensor.sum(axis=0)`

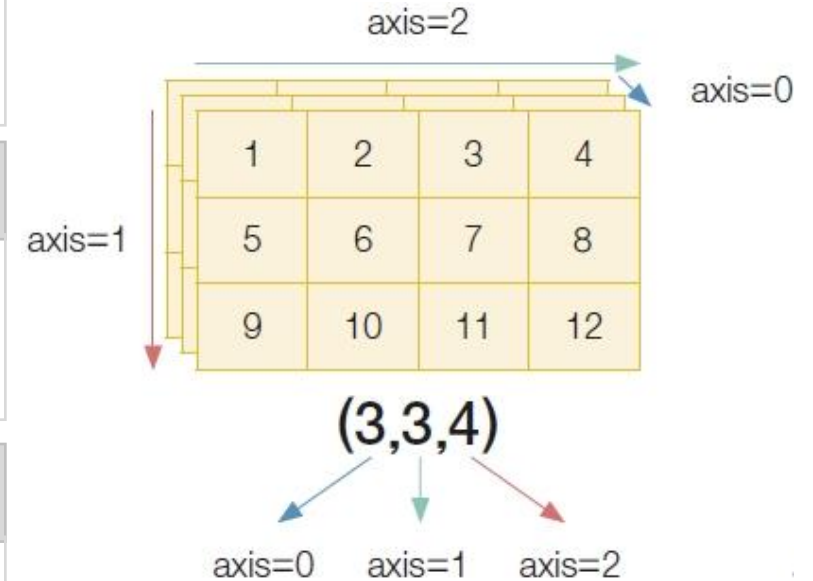
⇒ `array([[3, 6, 9, 12],
 [15, 18, 21, 24],
 [27, 30, 33, 36]])`

▶ `third_order_tensor.sum(axis=1)`

⇒ `array([[15, 18, 21, 24],
 [15, 18, 21, 24],
 [15, 18, 21, 24]])`

▶ `third_order_tensor.sum(axis=2)`

⇒ `array([[10, 26, 42],
 [10, 26, 42],
 [10, 26, 42]])`



5 연산 함수

- mean(), std(), sqrt() 함수

```
▶ test_array = np.arange(1, 13).reshape(3, 4)
test_array
```

```
⇒ array([[ 1,  2,  3,  4],
        [ 5,  6,  7,  8],
        [ 9, 10, 11, 12]])
```

```
▶ test_array.mean(axis=1) # axis=1 축을 기준으로 평균 연산
```

```
⇒ array([ 2.5,  6.5, 10.5])
```

```
▶ test_array.std() # 전체 값에 대한 표준편차 연산
```

```
⇒ np.float64(3.452052529534663)
```

```
▶ test_array.std(axis=0) # axis=0 축을 기준으로 표준편차 연산
```

```
⇒ array([3.26598632, 3.26598632, 3.26598632, 3.26598632])
```



5 연산 함수

- mean(), std(), sqrt() 함수

▶ np.sqrt(test_array) # 각 요소에 제곱근 연산 수행

```
↩ array([[1.          , 1.41421356, 1.73205081, 2.          ],
         [2.23606798, 2.44948974, 2.64575131, 2.82842712],
         [3.          , 3.16227766, 3.31662479, 3.46410162]])
```

5 배열 연산

■ 연결 함수

- 연결 함수(concatenation functions) : 두 객체 간의 결합을 지원하는 함수
- **vstack** 함수 : 배열을 수직으로 붙여 하나의 행렬을 생성
- **hstack** 함수 : 배열을 수평으로 붙여 하나의 행렬을 생성

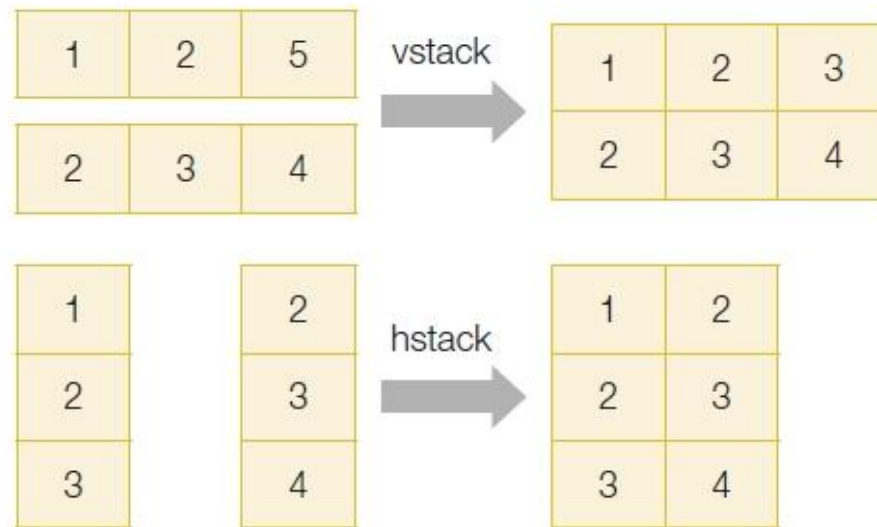


그림 3-14 연결을 통한 행렬의 결합

5 배열 연산

■ 연결 함수

- 넘파이는 열 벡터를 표현할 수 없어 2차원 행렬 형태로 표현

```
▶ v1 = np.array([1, 2, 3])
  v2 = np.array([4, 5, 6])
  np.vstack((v1, v2))
```

```
⇒ array([[1, 2, 3],
         [4, 5, 6]])
```

```
▶ np.hstack((v1,v2))
```

```
⇒ array([1, 2, 3, 4, 5, 6])
```

- 벡터 형태 그대로 **hstack**을 붙일 경우 그대로 벡터 형태의 배열 생성

5 배열 연산

■ 연결 함수

- 열벡터는 2차원 벡터로 표현 (3,1)

```
▶ v1 = v1.reshape(-1, 1) # v1 = np.array([1, 2, 3])
  v2 = v2.reshape(-1, 1) # v2 = np.array([4, 5, 6])
  v1
```

```
↪ array([[1],
        [2],
        [3]])
```

```
▶ v2
```

```
↪ array([[4],
        [5],
        [6]])
```

```
▶ np.hstack((v1,v2))
```

```
↪ array([[1, 4],
        [2, 5],
        [3, 6]])
```

- 2차원 행렬 형태로 표현한 열 벡터를 hstack으로 연결



5 배열 연산

■ 연결 함수

- **concatenate** 함수 : **축을 고려**하여 두 개의 배열을 결합
 - 스택(stack) 계열의 함수와 달리 생성될 배열과 소스가 되는 배열의 차원이 같아야 함
 - 두 벡터를 결합하고 싶다면, 해당 벡터를 일단 2차원 배열 꼴로 변환 후 행렬로 나타내야 함

```
▶ v1 = np.array([[1, 2, 3]])
  v2 = np.array([[4, 5, 6]])
  np.concatenate((v1,v2), axis=0)
```

```
⇒ array([[1, 2, 3],
         [4, 5, 6]])
```

- v1과 v2 모두 사실상 행렬이지만 벡터의 형태
- 매개변수 axis=0로 행을 기준으로 연결
- (참고)

```
▶ v1 = np.array([1, 2, 3])
  v2 = np.array([4, 5, 6])
  np.concatenate((v1,v2), axis=0)
```

```
⇒ array([1, 2, 3, 4, 5, 6])
```



5 배열 연산

■ 연결 함수

```
▶ v1 = np.array([1, 2, 3, 4]).reshape(2,2)
  v2 = np.array([[5,6]]).T
  v1
```

```
↪ array([[1, 2],
         [3, 4]])
```

```
▶ v2
```

```
↪ array([[5],
         [6]])
```

```
▶ np.concatenate((v1,v2), axis=1)
```

```
↪ array([[1, 2, 5],
         [3, 4, 6]])
```

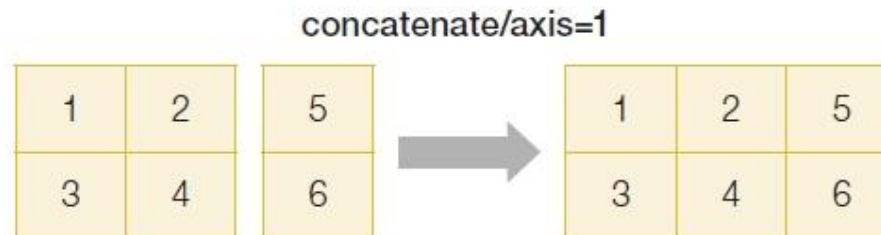


그림 3-15 배열의 연결



5 배열 연산

■ 사칙연산 함수

- 넘파이는 파이썬과 동일하게 배열 간 사칙연산 지원
 - 행렬과 행렬, 벡터와 벡터 간 사칙연산이 가능
- 같은 배열의 구조일 때 요소별 연산(element-wise operation)
 - 요소별 연산 : 두 배열의 구조가 동일할 경우 같은 인덱스 요소들끼리 연산

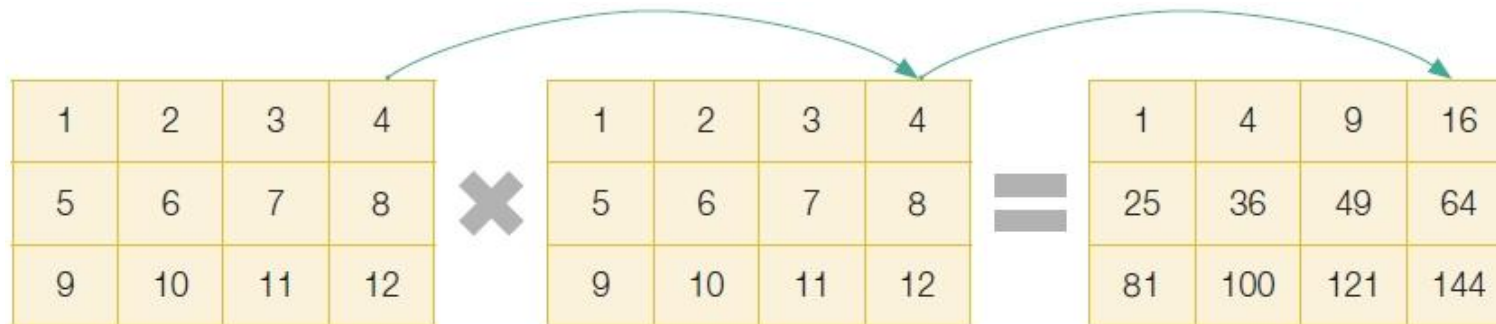


그림 3-16 배열 간 요소별 연산의 예시

5 배열 연산

■ 사칙연산 함수

▶ `x = np.arange(1, 7).reshape(2,3)`
x

⇒ `array([[1, 2, 3],
[4, 5, 6]])`

▶ `x + x`

⇒ `array([[2, 4, 6],
[8, 10, 12]])`

▶ `x / x`

⇒ `array([[1., 1., 1.],
[1., 1., 1.]])`

▶ `x - x`

⇒ `array([[0, 0, 0],
[0, 0, 0]])`

▶ `x ** x`

⇒ `array([[1, 4, 27],
[256, 3125, 46656]])`

5 배열 연산

■ 사칙연산 함수

- 배열 간의 곱셈에서는 요소별 연산과 벡터의 내적(dot product) 연산 가능
 - 벡터의 내적 : 두 배열 간의 곱셈
 - 두 개의 행렬에서 첫 번째 행렬의 열 크기와 두 번째 행렬의 행 크기가 동일해야 함
 - $m \times n$ 행렬과 $n \times l$ 행렬, 벡터의 내적 연산하면 $m \times l$ 의 행렬 생성

$$(m \times n) \cdot (n \times k) = (m \times k)$$

곱셈 연산이 정의됨

그림 3-17 벡터의 내적(dot product) 연산

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \cdot \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} (1 \cdot 5) + (2 \cdot 7) & (1 \cdot 6) + (2 \cdot 8) \\ (3 \cdot 5) + (4 \cdot 7) & (3 \cdot 6) + (4 \cdot 8) \end{bmatrix}$$

$$= \begin{bmatrix} 5 + 14 & 6 + 16 \\ 15 + 28 & 18 + 32 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

그림 3-18 벡터의 내적(dot product) 연산의 예시

5 배열 연산

■ 사칙연산 함수

- **dot** 함수 : 벡터의 **내적** 연산

```
▶ x_1 = np.arange(1, 7).reshape(2,3)
  x_2 = np.arange(1, 7).reshape(3,2)
  x_1
```

```
⇒ array([[1, 2, 3],
         [4, 5, 6]])
```



```
▶ x_2
```

```
⇒ array([[1, 2],
         [3, 4],
         [5, 6]])
```

```
▶ x_1.dot(x_2)
```

```
⇒ array([[22, 28],
         [49, 64]])
```

- 2×3 행렬과 3×2 행렬의 연산 결과는 2×2 행렬



5 배열 연산

■ 사칙연산 함수

- 브로드캐스팅 연산(broadcasting operations) :
하나의 행렬과 스칼라 값들 간의 연산이나 행렬과 벡터 간의 연산
 - 방송국의 전파가 퍼지듯 뒤에 있는 스칼라 값이 모든 요소에 퍼지듯이 연산

$$\begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline \end{array} + \begin{array}{|c|} \hline 3 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array}$$

그림 3-19 브로드캐스팅 연산

5 배열 연산

■ 사칙연산 함수

```
▶ x = np.arange(1, 10).reshape(3,3)
x
```

```
⇒ array([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
```

```
▶ x + 10
```

```
⇒ array([[11, 12, 13],
        [14, 15, 16],
        [17, 18, 19]])
```

```
▶ x - 2
```

```
⇒ array([[ -1,  0,  1],
        [  2,  3,  4],
        [  5,  6,  7]])
```

```
▶ x // 3
```

```
⇒ array([[0, 0, 1],
        [1, 1, 2],
        [2, 2, 3]])
```

```
▶ x ** 2
```

```
⇒ array([[ 1,  4,  9],
        [16, 25, 36],
        [49, 64, 81]])
```

5 배열 연산

■ 사칙연산 함수

- 행렬과 스칼라 값 외에 행렬과 벡터, 벡터와 벡터 간에도 연산

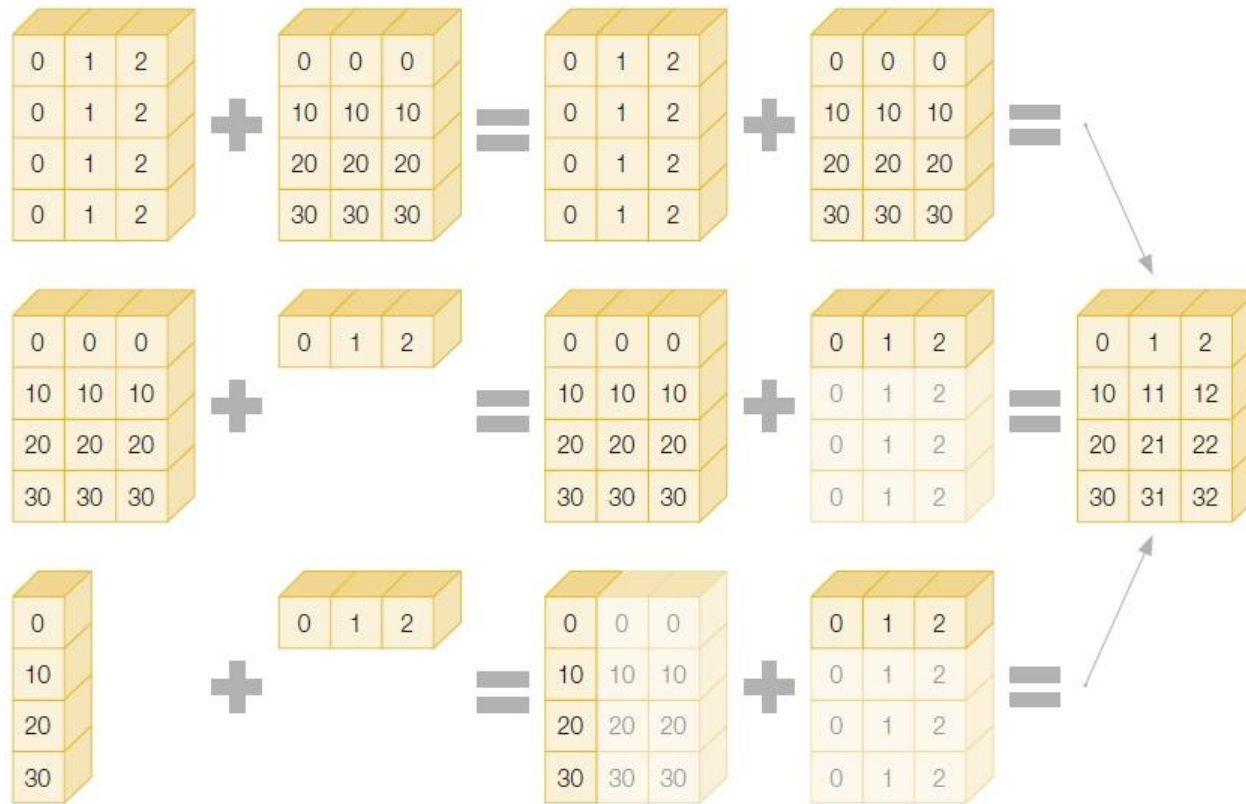


그림 3-20 확장된 브로드캐스팅 연산

5 배열 연산

■ 사칙연산 함수

```
▶ x = np.arange(1, 13).reshape(4,3)
x
```

```
↩ array([[ 1,  2,  3],
         [ 4,  5,  6],
         [ 7,  8,  9],
         [10, 11, 12]])
```

```
▶ v = np.arange(10, 40, 10)
v
```

```
↩ array([10, 20, 30])
```

```
▶ x + v
```

```
↩ array([[11, 22, 33],
         [14, 25, 36],
         [17, 28, 39],
         [20, 31, 42]])
```



5 배열 연산

■ 사칙연산 함수

1	2	3
4	5	6
7	8	9
10	11	12

+

10	20	30
----	----	----

=

11	22	33
14	25	36
17	28	39
20	31	42

그림 3-21 행렬과 벡터의 브로드캐스팅 연산

- 뒤에 있는 벡터가 앞에 있는 행렬과 크기를 맞추기 위해 4×3의 행렬처럼 복제
- 그 다음 요소별 연산처럼 연산



5 배열 연산

■ 비교 연산

- 연산 결과는 항상 불린형(boolean type)을 가진 배열로 추출

■ 브로드캐스팅 비교 연산

- 하나의 스칼라 값과 벡터 간의 비교 연산은 벡터 내 전체 요소에 적용

```
▶ import numpy as np
   x = np.array([4, 3, 2, 6, 8, 5])
   x > 3
```

```
⇒ array([ True, False, False,  True,  True,  True])
```



5 배열 연산

■ 요소별 비교 연산

- 두 개의 배열 간 배열의 구조(shape)가 동일한 경우
- 같은 위치에 있는 요소들끼리 비교 연산
- $[1 > 2, 3 > 1, 0 > 7]$ 과 같이 연산이 실시된 후 이를 반환

```
▶ x = np.array([1, 3, 0])
  y = np.array([2, 1, 7])
  x > y
```

```
⇒ array([False,  True, False])
```

5 배열 연산

■ 비교 연산 함수

- **all** 함수 : 배열 내부의 모든 값이 참일 때는 True, 하나라도 참이 아닐 경우에는 False를 반환
 - and 조건을 전체 요소에 적용
- **any** 함수 : 배열 내부의 값 중 하나라도 참일 때는 True, 모두 거짓일 경우 False를 반환
 - or 조건을 전체 요소에 적용

```
▶ x = np.array([4, 6, 7, 3, 2])  
(x > 3)
```

```
⇒ array([ True,  True,  True, False, False])
```

- $x > 3$ 브로드캐스팅이 적용되어 불린형으로 이루어진 배열 반환

```
▶ (x > 3).all()
```

```
⇒ np.False_
```

```
▶ (x > 3).any()
```

```
⇒ np.True_
```

- all 함수를 적용하면 2개의 거짓이 있기 때문에 False를 반환
- any 함수를 적용하면 참이 있기 때문에 True를 반환

5 배열 연산

▶ `(x < 10).all()`

↪ `np.True_`

▶ `(x > 10).any()`

↪ `np.False_`

- $x > 10$ 의 경우 모든 값이 10을 넘지 못하므로 모두 거짓
- `any` 함수를 적용해도 `False` 반환

5 배열 연산

■ 인덱스 반환 함수

- **where** 함수 : 배열이 불린형으로 이루어졌을 때 참인 값들의 인덱스를 반환

```
▶ x = np.array([4, 6, 7, 3, 2])  
  x > 5
```

```
⇒ array([False,  True,  True, False, False])
```

```
▶ np.where(x>5)
```

```
⇒ (array([1, 2]),)
```

- $x > 5$ 를 만족하는 값은 6과 7
- 6과 7의 인덱스 값인 [1, 2]를 반환

5 배열 연산

■ 인덱스 반환 함수

- True/False 대신 참/거짓인 경우의 값을 지정할 수 있음

```
▶ x = np.array([4, 6, 7, 3, 2])  
   np.where(x>5 , 10, 20)
```

```
↪ array([20, 10, 10, 20, 20])
```

- 참일 경우에 10을, 거짓일 경우에 20을 반환

5 배열 연산

■ 정렬된 값의 인덱스를 반환해주는 함수

- **argsort** : 배열 내 값들을 작은 순서대로 인덱스를 반환
- **argmax** : 배열 내 값들 중 가장 큰 값의 인덱스를 반환
- **argmin** : 배열 내 값들 중 가장 작은 값의 인덱스를 반환

```
▶ x = np.array([4, 6, 7, 3, 2])  
   np.argsort(x)
```

```
⇒ array([4, 3, 0, 1, 2])
```

```
▶ np.argmax(x)
```

```
⇒ np.int64(2)
```

```
▶ np.argmin(x)
```

```
⇒ np.int64(4)
```


5 배열 연산

■ 인덱스 활용한 데이터 추출

- 불린 인덱스(boolean index):
 - 배열에 있는 값들을 반환할 조건을 불린형의 배열에 넣어 추출
 - 인덱스에 들어가는 배열은 불린형이어야 함
 - 불린형 배열과 추출 대상이 되는 배열의 구조가 같아야 함

```
▶ x = np.array([4, 6, 7, 3, 2])
  x > 3
```

```
⇒ array([ True,  True,  True, False, False])
```

```
▶ cond = x > 3
  x[cond]
```

```
⇒ array([4, 6, 7])
```

```
▶ x.shape
```

```
⇒ (5,)
```

```
▶ cond.shape
```

```
⇒ (5,)
```

5 배열 연산

■ 인덱스 활용한 데이터 추출

- 팬시 인덱스(fancy index) :
 - 정수형 배열의 값을 사용하여 해당 정수의 인덱스에 위치한 값을 반환
 - 인덱스 항목에 넣을 배열은 정수로만 구성되어야 함
 - 정수 값의 범위는 대상이 되는 배열이 가지는 인덱스의 범위 내 대상이 되는 배열과 인덱스 배열의 구조(shape)가 같을 필요는 없음

```
▶ x = np.array([4, 6, 7, 3, 2])
  cond = np.array([1, 2, 0, 2, 2, 2,], int)
  x[cond]
```

```
⇒ array([6, 7, 4, 7, 7, 7])
```

```
▶ x.take(cond)
```

```
⇒ array([6, 7, 4, 7, 7, 7])
```

5 배열 연산

■ 인덱스 활용한 데이터 추출

```
▶ x = np.array([[1,4], [9,16]], int)
  a = np.array([0, 1, 1, 1, 0, 0], int)
  b = np.array([0, 0, 0, 1, 1, 1], int)
  x[a,b]
```

```
↪ array([ 1,  9,  9, 16,  4,  4])
```

	0	1
0	1	4
1	9	16

그림 3-23 팬시 인덱스



Q & A

